



36-MAVZU · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django Models va ORM texnologiyasi. Model yaratish va migratsiyalar

Model — jadvalning Python klassi, maydon turlari, ForeignKey bog'lanishlari, __str__ va Meta; makemigrations va migrate; ORM bilan CRUD: create, all, filter, get, order_by, update, delete; get_object_or_404; view'larni bazaga ulash.

Bugun nimani o'rganamiz?

- **Model** — jadvalni Python klassi sifatida yozish
- Maydon turlari va **ForeignKey** bog'lanishlari
- **Migratsiya** — modeldan haqiqiy jadvalga
- **ORM**: SQL yozmasdan CRUD va filtrlash
- Blog sahifalarini haqiqiy **PostgreSQL**'ga ulash

ORM nima?

ORM — jadvallar = klasslar, qatorlar = obyektlar, SQL o'rniga Python

SQL	Django ORM
<code>SELECT * FROM blog_maqola;</code>	<code>Maqola.objects.all()</code>
<code>... WHERE id = 5;</code>	<code>Maqola.objects.get(id=5)</code>
<code>... WHERE sarlavha ILIKE '%futbol%'</code>	<code>.filter(sarlavha__icontains="futbol")</code>
<code>INSERT INTO ... VALUES ...</code>	<code>Maqola.objects.create(...)</code>



Tarjimon: siz Python'da gapirasiz, baza faqat SQL tushunadi. ORM to'g'ri va **xavfsiz** SQL'ga o'giradi — SQL injection'dan avtomatik himoya!

Model yaratish: Kategoriya

blog/models.py

```
from django.db import models
from django.contrib.auth.models import User
from django.urls import reverse

class Kategoriya(models.Model):
    nomi = models.CharField("Nomi", max_length=100, unique=True)
    slug = models.SlugField(unique=True)

    def __str__(self):
        return self.nomi
```

- Har bir model — `models.Model` dan meros (25-mavzu!); har atribut — ustun; `id` avtomatik

Model yaratish: Maqola

blog/models.py

```
class Maqola(models.Model):
    sarlavha = models.CharField("Sarlavha", max_length=200)
    matn = models.TextField("Matn")
    kategoriya = models.ForeignKey(Kategoriya, on_delete=models.PROTECT, related_name="maqolalar")
    muallif = models.ForeignKey(User, on_delete=models.CASCADE)
    chop_etilgan = models.BooleanField(default=True)
    korishlar = models.PositiveIntegerField(default=0)
    yaratilgan = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-yaratilgan"]

    def __str__(self):
        return self.sarlavha

    def get_absolute_url(self):
        return reverse("blog:maqola", args=[self.id])
```

Maydon turlari

Django maydoni	PostgreSQL	Qachon
<code>CharField(max_length)</code>	VARCHAR	Ism, sarlavha
<code>TextField()</code>	TEXT	Uzun matn
<code>IntegerField()</code>	INTEGER	Butun son
<code>DecimalField()</code>	NUMERIC	Pul
<code>BooleanField()</code>	BOOLEAN	Ha/yo'q
<code>.DateField()</code> , <code>DateTimeField()</code>	DATE, TIMESTAMP	Sana, vaqt
<code>EmailField()</code> , <code>SlugField()</code>	VARCHAR + tekshiruv	Email, slug
<code>ImageField(upload_to)</code>	VARCHAR (yo'l)	Rasm (Pillow)
<code>ForeignKey(Model, on_delete)</code>	FK <code>..._id</code>	1:N
<code>ManyToManyField(Model)</code>	Oraliq jadval	N:M

Parametrlar va on_delete

- `null=True` — bazada NULL; `blank=True` — formada bo'sh qoldirish mumkin
- `default=...`, `unique=True`, `choices=[...]`
- Birinchi argument (`"Sarlavha"`) — admin va formalardagi chiroyli nom
- `id` yozilmaydi — Django avtomatik qo'shadi

Django	SQL	Misol
<code>CASCADE</code>	ON DELETE CASCADE	Foydalanuvchi o'chsa — maqolalari ham
<code>PROTECT</code>	RESTRICT	Maqolasi bor kategoriya o'chmaydi
<code>SET_NULL</code>	SET NULL (<code>null=True</code>)	—

Migratsiyalar — modeldan jadvalga

Terminal

```
python manage.py makemigrations    # 1) o'zgarishlardan "reja" (migratsiya fayli)
python manage.py migrate           # 2) rejani bazaga qo'llash (CREATE TABLE)
```

Natija

```
Migrations for 'blog':
  blog/migrations/0001_initial.py
    - Create model Kategoriya
    - Create model Maqola
Applying blog.0001_initial... OK
```



Qurilish chizmasi: `models.py` — yangi loyiha; `makemigrations` — ish rejasini yozish; `migrate` — rejani bajarish. Migratsiyalar — o'zgarishlar tarixi (Git kabi!)

- pgAdmin'da: `blog_kategoriya`, `blog_maqola` (`kategoriya_id`, `muallif_id` FK)



Har `models.py` o'zgarishidan keyin — **`makemigrations + migrate`**

ORM: Create

Terminal

```
python manage.py createsuperuser  
python manage.py shell
```

Python

```
from blog.models import Kategoriya, Maqola  
from django.contrib.auth.models import User  
  
admin = User.objects.first()  
sport = Kategoriya.objects.create(nomi="Sport", slug="sport")  
  
m = Maqola.objects.create(  
    sarlavha="Futbol bo'yicha g'alaba",  
    matn="Maktabimiz jamoasi shahar chempionatida g'olib bo'ldi!",  
    kategoriya=sport,  
    muallif=admin,  
)  
print(m.id, m.yaratilgan)
```

ORM: Read

Python

```
Maqola.objects.all()           # barchasi (QuerySet)
Maqola.objects.count()
Maqola.objects.get(id=1)       # aynan bittasi
Maqola.objects.filter(chop_etilgan=True)
Maqola.objects.filter(kategoriya__slug="sport")   # JOIN!
Maqola.objects.filter(sarlavha__icontains="futbol") # ILIKE
Maqola.objects.filter(korishlar__gte=100)         # ≥ 100
Maqola.objects.exclude(chop_etilgan=False)       # NOT
Maqola.objects.order_by("-korishlar")[:5]        # DESC LIMIT 5

m.kategoriya.nomi              # FK orqali bog'langan obyekt
sport.maqolalar.all()         # teskari yo'nalish (related_name)
```

Lookup'lar

Lookup	SQL
maydon=x	= x
<code>__gt</code> / <code>__gte</code> / <code>__lt</code> / <code>__lte</code>	> ≥ < ≤
<code>__contains</code> / <code>__icontains</code>	LIKE / ILIKE '%x%'
<code>__startswith</code>	LIKE 'x%'
<code>__in=[1, 2]</code>	IN (1, 2)
<code>__isnull=True</code>	IS NULL
fk__maydon	JOIN orqali bog'langan ustun

Update, Delete va agregatsiya

Python

```
m = Maqola.objects.get(id=1)
m.sarlavha = "Futbol: shahar chempionlari!"
m.save() # UPDATE

Maqola.objects.filter(kategoriya__slug="sport").update(chop_etilgan=False) # ko'pini birdan
Maqola.objects.get(id=2).delete() # DELETE

from django.db.models import Count, Sum, Avg
Maqola.objects.aggregate(jami=Sum("korishlar"), ortacha=Avg("korishlar"))
for k in Kategoriya.objects.annotate(soni=Count("maqolalar")): # GROUP BY
    print(k.nomi, k.soni)
```

- QuerySet "**dangasa**": SQL faqat natija kerak bo'lganda yuboriladi

View'larni bazaga ulash

blog/views.py

```
from django.shortcuts import render, get_object_or_404
from django.db.models import Q, F
from .models import Maqola, Kategoriya

def bosh_sahifa(request):
    maqolalar = Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya", "muallif")
    return render(request, "blog/bosh_sahifa.html", {"maqolalar": maqolalar})

def maqola(request, id):
    m = get_object_or_404(Maqola, id=id, chop_etilgan=True) # topilmasa - 404
    Maqola.objects.filter(id=m.id).update(korishlar=F("korishlar") + 1)
    return render(request, "blog/maqola.html", {"maqola": m})

def qidiruv(request):
    q = request.GET.get("q", "").strip()
    natija = Maqola.objects.filter(Q(sarlavha__icontains=q) | Q(matn__icontains=q)) if q else []
    return render(request, "blog/qidiruv.html", {"q": q, "maqolalar": natija})
```

- `select_related` — bitta JOIN bilan; usiz "N+1 muammosi" (10 maqola = 21 so'rov)

Shablon — model atributlari bilan

blog/_karta.html

```
<article class="karta">
  <span class="belgi">{{ m.kategoriya.nomi }}</span>
  <h3><a href="{{ m.get_absolute_url }}">{{ m.sarlavha }}</a></h3>
  <p>{{ m.matn|truncatewords:20 }}</p>
  <small>👉 {{ m.muallif.username }} · 📅 {{ m.yaratilgan|date:"d.m.Y" }} · 👁 {{ m.korishlar }}</small>
</article>
```

Amaliyot

- 1 `Kategoriya` va `Maqola` modellari → `makemigrations` + `migrate`
- 2 `createsuperuser`; shell'da 3 kategoriya va 6 maqola
- 3 ORM mashqlari: sport maqolalari, "maktab" so'zi borlar, top-3 ko'rilgan
- 4 View'larni bazaga ulang; `kategoriya.html`, `qidiruv.html` + qidiruv formasi
- 5 Bosh sahifada kategoriyalar va maqolalar soni (`annotate(Count(...))`)
- 6 `rasm = models.ImageField(upload_to="maqolalar/", blank=True)` + `MEDIA_URL`,
`MEDIA_ROOT`

Ko'p uchraydigan xatolar

- ❌ `relation "blog_maqola" does not exist` — `migrate` bajarilmagan
- ❌ `No changes detected` — ilova `INSTALLED_APPS` da yo'q
- ❌ `Maqola matching query does not exist` — `get_object_or_404` ishlating
- ❌ `get() returned more than one` — ko'p natija uchun `filter`
- ❌ Yangi majburiy maydon → `default=...` yoki `null=True`

Savol

ORM nima?

- A** Ma'lumotlar bazasi turi
- B** Jadvallar bilan klass va obyektlar orqali ishlash texnologiyasi
- C** HTML shablon tili
- D** Server dasturi

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

Modelni o'zgartirgandan keyin bajariladigan ikki buyruq?

A runserver va shell

B startapp va migrate

C makemigrations va migrate

D createsuperuser va collectstatic

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

Sarlavhasida "olimpiada" bor maqolalar (katta-kichik harfga qaramay)?

- A** `Maqola.objects.get(sarlavha="olimpiada")`
- B** `Maqola.objects.all("olimpiada")`
- C** `.filter(sarlavha="%olimpiada%")`
- D** `.filter(sarlavha__icontains="olimpiada")`

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya?

A `Maqola.objects.get()`

B `get_object_or_404()`

C `find_or_404()`

D `Http404()`

Javobni tanlang — to'g'ri variant yashil bo'ladi



Uyga vazifa

- 1 **Kutubxona (22-loyiha):** `Muallif`, `Kitob` (FK, janr — choices), `Oquvchi` modellari
- 2 **Teglar:** `Teg` modeli + `Maqola.teglar = models.ManyToManyField(Teg, blank=True)`

✓ Yodda tuting

- `class X(models.Model); maydonlar — ustunlar; __str__, Meta, get_absolute_url`
- `ForeignKey(..., on_delete=..., related_name=...), ManyToManyField`
- **makemigrations → migrate**
- `create, all, get, filter(maydon__lookup=...), order_by, save, delete, annotate`
- View'larda `get_object_or_404, select_related`



Savollar bormi?

Keyingi mavzu: **37. Django Admin panel bilan ishlash. Django Forms yordamida ma'lumotlarni qabul qilish**

Batafsil matn, misollar va testlar: **36-django-model-orm.html** darsida